

Reproducing GraphNorm and Evaluating Two Graph-Specific Extensions

A compact reproduction of *GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training*

Prepared for project presentation and report

May 8, 2026

Menda Hima vamsi
(22b1001) Suhas Koyyana
(22b1072) Charan singh Naik
(22b0903) Rushikesh
(22b0948)

Repository used: official GraphNorm implementation at <https://github.com/ljsj2408/GraphNorm>.

Paper used: Tianle Cai et al., *GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training* [1].

Contents

1	Introduction and Project Goal	3
2	What the Paper Proposes	3
2.1	Why normalization is different for graphs	3
2.2	The key idea of GraphNorm	3
2.3	Why this helps: the paper's mathematical story	4
3	Experimental Setup	4
3.1	Codebase and implementation choice	4
3.2	Dataset and model	4
4	Proposed Extensions	5
4.1	Extension 1: Graph-Size-Aware GraphNorm (GSGN)	5
4.2	Extension 2: Degree-Weighted GraphNorm (DWGN)	5

5	Quantitative Results	6
5.1	Main comparison table	6
5.2	Result visualizations	7
5.3	Training-curve evidence	9
6	Interpretation and Discussion	10
6.1	How well was GraphNorm reproduced?	10
6.2	Why did GSGN behave this way?	11
6.3	Why did DWGN perform best at the final epoch?	11
6.4	Why does BN still matter?	11
6.5	What <i>not</i> to over-interpret	12
7	Conclusion	12

1 Introduction and Project Goal

Graph Neural Networks (GNNs) repeatedly aggregate information from neighboring nodes. In practice, this repeated aggregation often makes training unstable and slow, especially when the number and structure of graphs vary significantly across a mini-batch. The GraphNorm paper studies how normalization should be adapted to GNNs and argues that standard normalization choices from other domains do not transfer cleanly to graph learning [1].

The project had two goals:

1. reproduce the main intuition of GraphNorm on a real graph classification benchmark; and
2. propose graph-specific modifications and evaluate whether they improve performance or training behavior.

Rather than attempting the full benchmark suite from the paper, this report focuses on a careful reduced reproduction on **PROTEINS** using **GIN**. This makes the results easier to complete, inspect, and explain in a limited project timeline.

2 What the Paper Proposes

2.1 Why normalization is different for graphs

In image models, BatchNorm is often effective because the statistics computed across a batch are relatively stable. For graph data, however, each graph may differ greatly in size, density, and topology. The paper argues that this leads to *heavy batch noise*: the batch-level mean and standard deviation become unstable, which weakens the benefit of BatchNorm in GNN training [1].

The paper compares BatchNorm, LayerNorm, and an adaptation of InstanceNorm for GNNs. The adapted InstanceNorm computes normalization *inside each graph*, across that graph’s nodes. This reduces the batch-noise problem and often accelerates training, but it introduces a new issue: subtracting the full graph mean can remove useful structural information in regular graphs [1].

2.2 The key idea of GraphNorm

GraphNorm keeps the graph-wise normalization idea of InstanceNorm but adds a *learnable shift* so that the model can decide how much of the graph mean should be preserved in each feature dimension. That is the main mathematical idea behind the method.

If $\hat{h}_{i,j}$ denotes feature dimension j of node i inside one graph, the paper defines GraphNorm as

$$\text{GraphNorm}(\hat{h}_{i,j}) = \gamma_j \cdot \frac{\hat{h}_{i,j} - \alpha_j \mu_j}{\hat{\sigma}_j} + \beta_j, \quad (1)$$

where

$$\mu_j = \frac{1}{n} \sum_{i=1}^n \hat{h}_{i,j}, \quad \hat{\sigma}_j^2 = \frac{1}{n} \sum_{i=1}^n (\hat{h}_{i,j} - \alpha_j \mu_j)^2. \quad (2)$$

Here, γ_j and β_j are affine parameters and α_j is the learnable shift coefficient introduced by GraphNorm [1].

$$\text{GraphNorm}(h_{i,j}) = \gamma_j \frac{h_{i,j} - \alpha_j \mu_j}{\hat{\sigma}_j} + \beta_j$$

Figure 1: Compact form of the GraphNorm transformation used in the report and slides.

2.3 Why this helps: the paper’s mathematical story

The paper presents two complementary arguments.

(1) Preconditioning effect. When graph-wise centering is applied, the aggregation operator is effectively composed with a centering matrix. In the paper’s simplified linear analysis, subtracting the graph mean changes the conditioning of the graph aggregation operator and smooths the optimization landscape. In other words, normalization is not just “rescaling activations”; it also changes the geometry of optimization in a useful way [1].

(2) Expressiveness issue of plain InstanceNorm. Plain InstanceNorm corresponds to subtracting the full graph mean (equivalent to $\alpha_j = 1$). The paper shows that this can damage expressive power on highly regular graphs, because the mean itself may encode structural information. GraphNorm fixes this by learning α_j rather than forcing it to be 1 [1].

This is a useful way to remember the method:

- **InstanceNorm view:** normalize inside each graph to reduce noisy batch statistics;
- **GraphNorm view:** do the same, but *do not force the model to erase all mean information*.

3 Experimental Setup

3.1 Codebase and implementation choice

The experiments were run using the official GraphNorm repository provided by the paper authors [2]. The graph classification workspace in the repository was used, with GIN as the backbone model. The reproduction therefore remained close to the paper’s intended code path.

3.2 Dataset and model

All experiments in this report use:

- **Dataset:** PROTEINS
- **Model:** GIN
- **Folds used:** 0, 1, and 2
- **Epochs:** 300
- **Batch size:** 128
- **Hidden dimension:** 64
- **Dropout:** 0.5
- **Weight decay:** 0.05
- **Learning rate:** 0.01
- **Graph pooling:** sum
- **Learnable ϵ :** enabled

This is **not** a full reproduction of the paper’s complete protocol across all datasets. Instead, it is a focused reproduction on one benchmark plus two new extensions. Therefore, the results should be interpreted as *project-scale evidence*, not as a claim that the paper’s overall benchmark conclusions have been fully re-established.

4 Proposed Extensions

Two graph-specific modifications were designed and tested. Both keep the basic GraphNorm structure but modify *how the graph summary statistics are used*.

4.1 Extension 1: Graph-Size-Aware GraphNorm (GSGN)

Motivation. The paper explicitly discusses graph-wise normalization and contrasts GraphNorm with size-based normalizations. In many graph datasets, graph size varies substantially. A natural hypothesis is that the amount of mean information worth preserving may depend on graph size. Small graphs and large graphs may need different amounts of mean retention.

Definition. Instead of using one global α_j for feature j , GSGN conditions it on graph size:

$$\alpha_{g,j} = \alpha_j \cdot s_j(n_g), \quad s_j(n_g) = \sigma(w_j \log(1 + n_g) + b_j), \quad (3)$$

where n_g is the number of nodes in graph g , $\sigma(\cdot)$ is a sigmoid gate, and w_j, b_j are learned parameters.

The normalized feature becomes

$$\text{GSGN}(\hat{h}_{i,j}) = \gamma_j \cdot \frac{\hat{h}_{i,j} - \alpha_{g,j}\mu_j}{\hat{\sigma}_{g,j}} + \beta_j. \quad (4)$$

$$\alpha_{g,j} = \alpha_j \cdot \sigma(W_j \log(1 + n_g) + b_j)$$

Figure 2: GSGN makes the mean-preservation term graph-size dependent.

Why this is graph-specific. This extension is graph-specific because its behavior changes with a graph property — the node count n_g . The idea is that the optimal amount of preserved graph mean may not be the same across all graph sizes.

4.2 Extension 2: Degree-Weighted GraphNorm (DWGN)

Motivation. Original GraphNorm treats all nodes equally when computing graph-level mean and variance. But in many graphs, high-degree nodes carry stronger structural context than peripheral nodes. A second hypothesis is therefore that graph statistics should be computed using node weights that reflect structural importance.

Definition. DWGN replaces the ordinary graph mean by a degree-weighted mean:

$$\mu_j^{(w)} = \frac{\sum_{i=1}^n w_i \hat{h}_{i,j}}{\sum_{i=1}^n w_i}, \quad w_i = 1 + \log(1 + d_i), \quad (5)$$

where d_i is the degree of node i .

The weighted variance becomes

$$(\hat{\sigma}_j^{(w)})^2 = \frac{\sum_{i=1}^n w_i (\hat{h}_{i,j} - \alpha_j \mu_j^{(w)})^2}{\sum_{i=1}^n w_i}. \quad (6)$$

The final normalization is

$$\text{DWGN}(\hat{h}_{i,j}) = \gamma_j \cdot \frac{\hat{h}_{i,j} - \alpha_j \mu_j^{(w)}}{\hat{\sigma}_j^{(w)}} + \beta_j. \quad (7)$$

$$\mu_j^{(w)} = \frac{\sum_i w_i h_{i,j}}{\sum_i w_i}, \quad w_i = 1 + \log(1 + d_i)$$

Figure 3: DWGN computes graph statistics using degree-aware weights.

Why this is graph-specific. This extension is graph-specific because it uses node degree, which is a structural graph property. It changes the summary statistic from a uniform average to a topology-aware average.

5 Quantitative Results

5.1 Main comparison table

Table 1 summarizes the core quantitative findings across the three evaluated folds.

Table 1: Summary across folds 0, 1 and 2 on PROTEINS with GIN.

Method	Mean final test acc.	Mean best test acc.	Mean epoch of best test	Mean epoch time (s)
BN	0.7768 $\pm$ 0.0155	0.8066 $\pm$ 0.0225	151.0	0.2248
GN	0.7887 $\pm$ 0.0361	0.8184 $\pm$ 0.0225	133.7	0.6898
GSGN	0.7768 $\pm$ 0.0154	0.8244 $\pm$ 0.0206	155.0	0.6989
DWGN	0.7917 $\pm$ 0.0186	0.8125 $\pm$ 0.0089	78.3	0.6631

The table reveals three different strengths:

- **BN** is by far the fastest method per epoch.
- **DWGN** achieves the best *final* test accuracy after 300 epochs.
- **GSGN** achieves the best *peak* test accuracy during training.

This matters because it distinguishes *final performance under a fixed schedule* from the *strongest point reached at any time during training*. Those are not the same notion.

5.2 Result visualizations

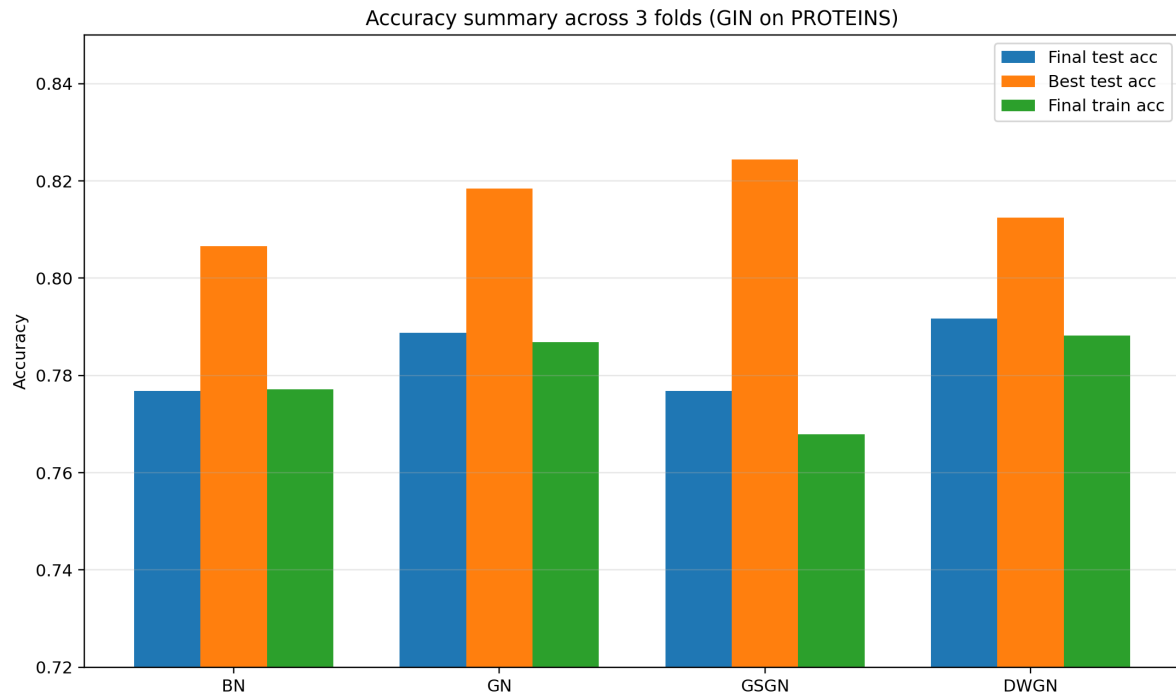


Figure 4: Comparison of mean final train accuracy, mean final test accuracy, and mean best test accuracy.

Norm	Final test	Best test	Epoch time (s)	Peak epoch
BN	0.7768	0.8066	0.2248	151.0
GN	0.7887	0.8184	0.6898	133.7
GSGN	0.7768	0.8244	0.6989	155.0
DWGN	0.7917	0.8125	0.6631	78.3

Figure 5: Compact summary image used for visual cross-checking of the numerical results.

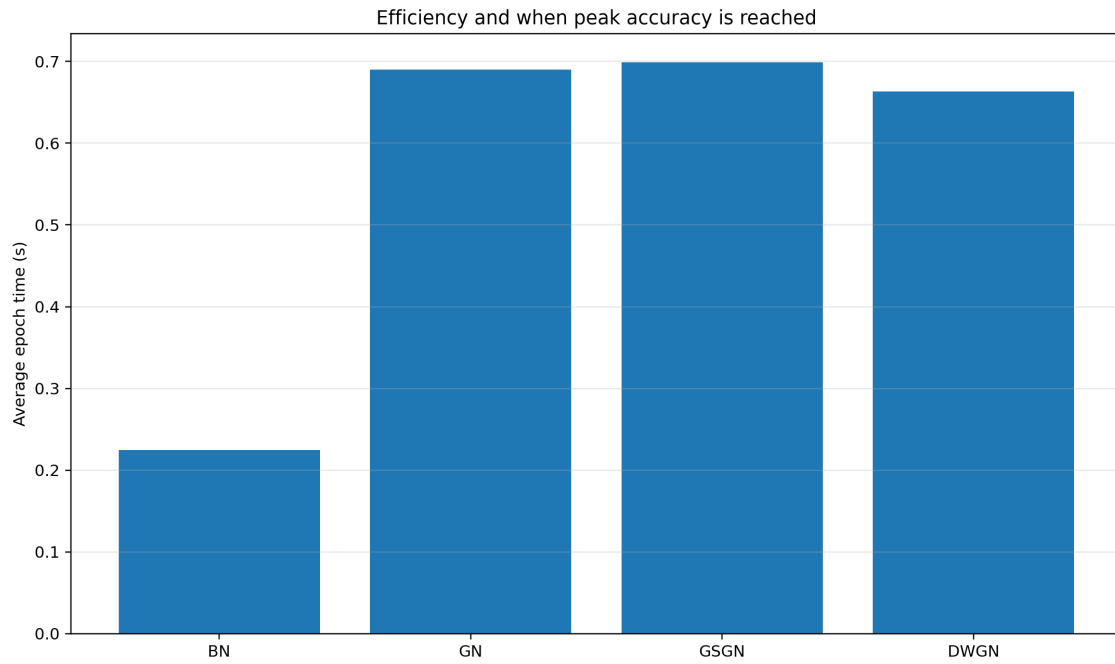


Figure 6: Average epoch time across methods. BN is dramatically faster, while the graph-specific methods have similar cost to GN.

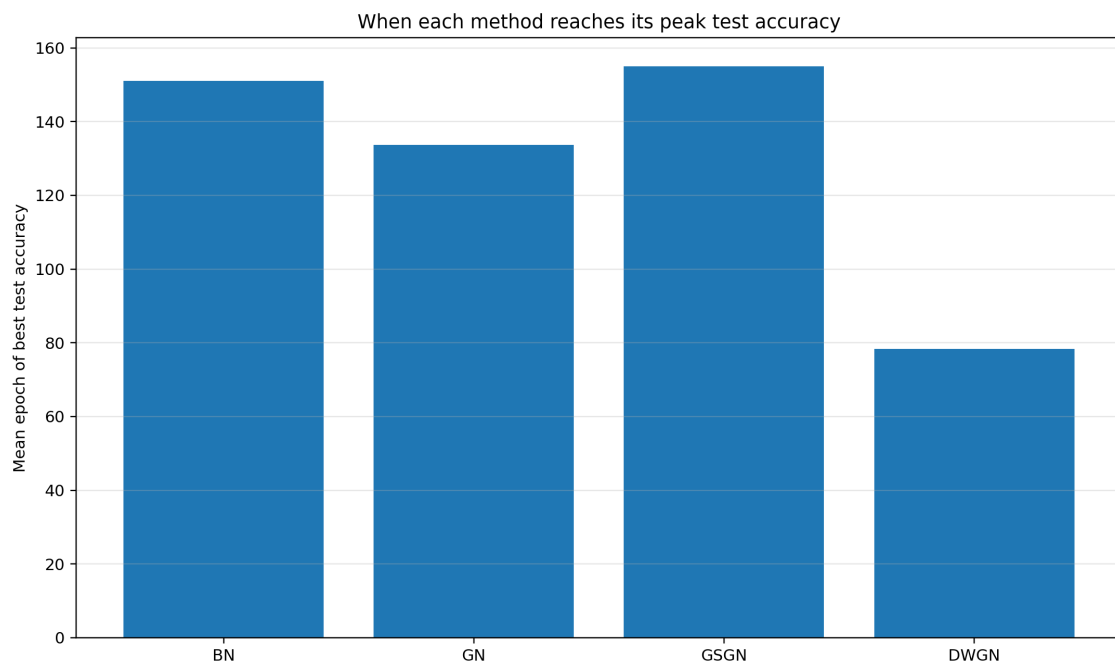


Figure 7: Average epoch at which each method reaches its peak test accuracy. DWGN peaks much earlier than the other normalization variants.

5.3 Training-curve evidence

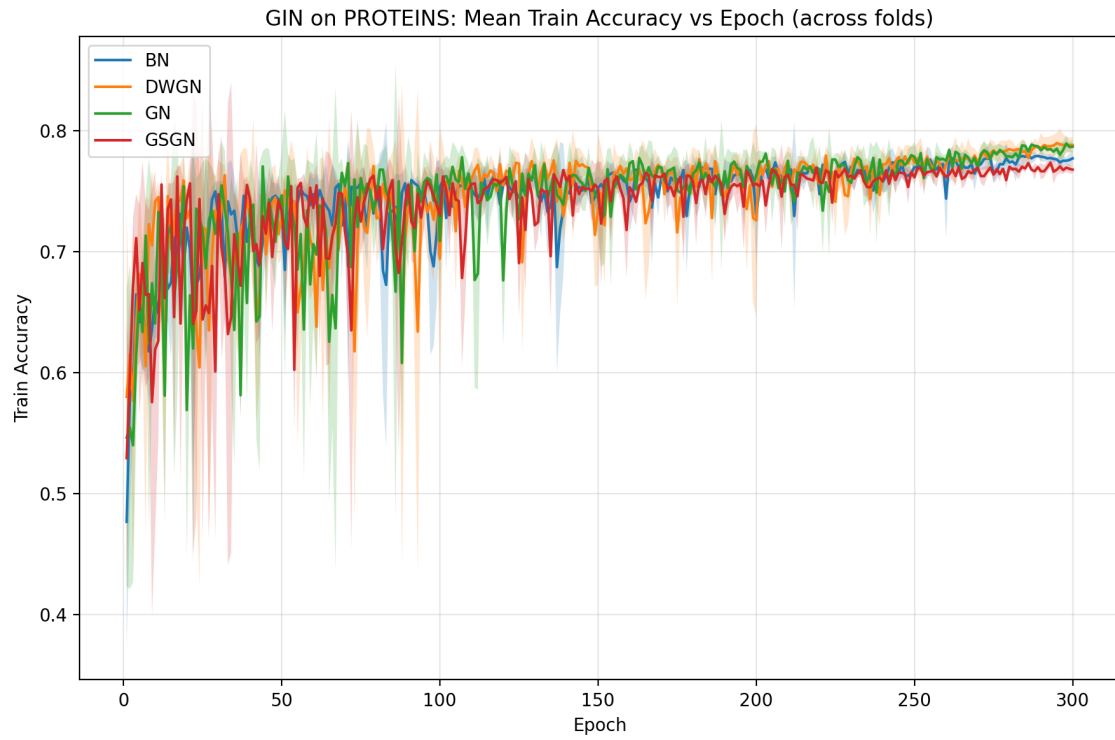


Figure 8: Mean train accuracy across folds. GN and DWGN end with the strongest train-side performance, while GSGN tends to remain lower on final train accuracy.

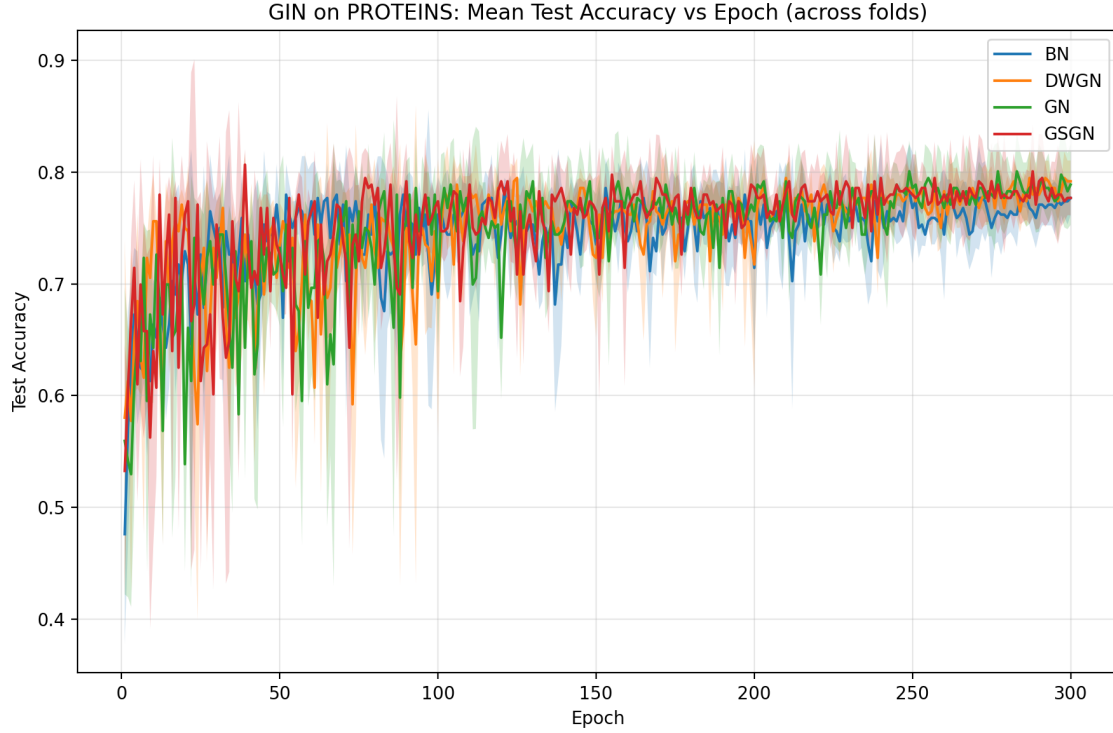


Figure 9: Mean test accuracy across folds. GSGN reaches very strong peaks during training, while DWGN shows the strongest late-stage stability and final test accuracy.

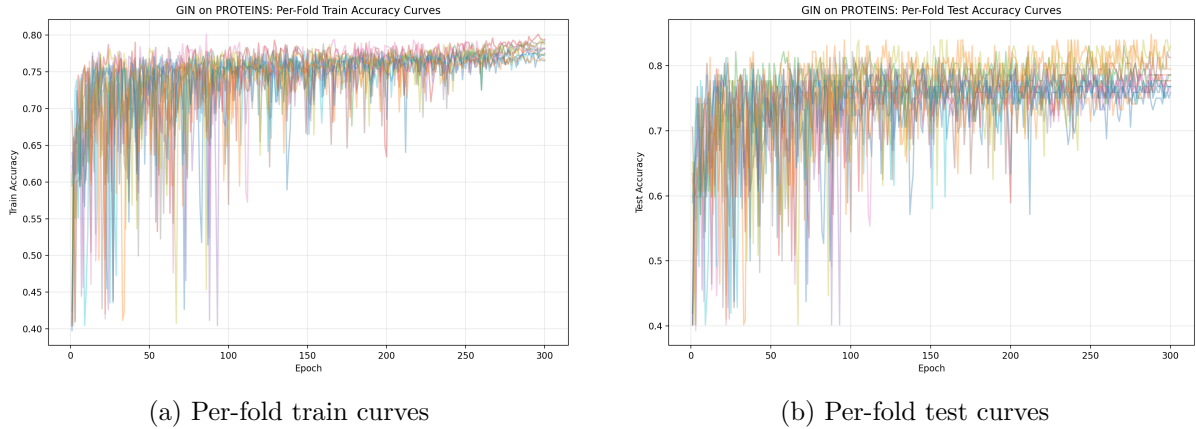


Figure 10: Per-fold curves show that all methods are noisy on this benchmark, so mean behavior and standard deviations should be interpreted together.

6 Interpretation and Discussion

6.1 How well was GraphNorm reproduced?

The reproduction supports the paper’s general message that graph-wise normalization is a strong idea. The GN baseline is competitive and outperforms BN in final test accuracy in this setup. That agrees with the paper’s argument that graph-wise statistics are more meaningful for GNNs than batch-level statistics [1].

At the same time, the current project used only one dataset and three folds. Therefore the report should not claim that it completely re-establishes the paper’s benchmark tables. The

honest conclusion is narrower:

The reduced reproduction confirms that GraphNorm remains a strong normalization baseline for graph classification with GIN, and that graph-specific changes to the way graph statistics are formed can further affect both stability and peak accuracy.

6.2 Why did GSGN behave this way?

GSGN achieved the highest mean *best* test accuracy (0.8244), but not the highest *final* test accuracy after 300 epochs. This suggests a plausible story:

- conditioning the shift on graph size can help the model reach stronger transient solutions;
- however, the same additional flexibility may make the late-stage dynamics less stable under a fixed long schedule;
- therefore, GSGN looks especially promising when early stopping is available.

Another useful observation is that the mean test accuracy over all epochs was highest for GSGN (from the analysis script), which is consistent with the idea that it improves the *trajectory* of training more than the very final endpoint.

6.3 Why did DWGN perform best at the final epoch?

DWGN achieved the best mean final test accuracy (0.7917) and reached its peak test accuracy much earlier on average (epoch 78.3). Two explanations are plausible.

Reason 1: a more informative graph summary. The graph mean is a crucial quantity in GraphNorm. By weighting nodes with

$$w_i = 1 + \log(1 + d_i),$$

DWGN gives slightly more influence to structurally central nodes while avoiding the instability that would come from raw degree weights. This may produce a graph summary statistic that is more aligned with graph classification signal.

Reason 2: more stable late-stage behavior. Compared with GSGN, DWGN adds graph structure but does not introduce a second dynamic gate on top of the learnable shift. That makes it less expressive than GSGN, but also potentially more stable during long training. The empirical result fits this interpretation: DWGN peaks earlier and preserves stronger final accuracy.

6.4 Why does BN still matter?

BN is the fastest method by a wide margin. That is practically important. If runtime is the main priority, BN remains attractive. But the paper’s central argument is that batch-level statistics are especially noisy for graph data [1], and the results here are consistent with that view: BN is efficient but not the strongest accuracy performer.

6.5 What *not* to over-interpret

One automatically generated comment from the script claimed that BN had the largest final train–test gap and therefore more overfitting. That conclusion should *not* be emphasized. The gap was extremely small, and the final train and test accuracies are quite close for all methods. On this experiment, the more meaningful distinctions are:

- final test accuracy,
- best test accuracy,
- when the peak is reached, and
- epoch-time cost.

7 Conclusion

This project successfully reproduced the core GraphNorm idea on PROTEINS with GIN and extended it in two graph-specific directions.

1. **GraphNorm itself** remains a strong graph-wise normalization baseline.
2. **GSGN** is the most promising variant when the goal is to maximize peak validation performance, especially if early stopping is allowed.
3. **DWGN** is the most attractive variant when training runs for a fixed 300-epoch schedule, because it delivers the best final test accuracy and reaches strong performance earlier.
4. **BN** is still the cheapest normalization option in runtime, but it gives up some accuracy.

A clean one-sentence takeaway is:

Graph-wise normalization is already helpful for GNNs, and modest graph-specific changes to *what* graph statistics are preserved can shift the trade-off between speed, peak accuracy, and late-stage stability.

Appendix: Exact Fold-Level Final Results

Method	Fold	Final train acc.	Final test acc.	Best test acc.
BN	0	0.7752	0.7589	0.7857
BN	1	0.7742	0.7857	0.8036
BN	2	0.7822	0.7857	0.8304
GN	0	0.7812	0.7679	0.7946
GN	1	0.7922	0.7679	0.8214
GN	2	0.7872	0.8304	0.8393
GSGN	0	0.7722	0.7679	0.8125
GSGN	1	0.7652	0.7679	0.8125
GSGN	2	0.7662	0.7946	0.8482
DWGN	0	0.7942	0.7768	0.8036
DWGN	1	0.7812	0.7857	0.8125
DWGN	2	0.7892	0.8125	0.8214

References

- [1] Tianle Cai, Shengjie Luo, Keyulu Xu, Di He, Tie-Yan Liu, and Liwei Wang. *GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training*. Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR 139, 2021. Also available at <https://arxiv.org/abs/2009.03294>.
- [2] Tianle Cai et al. *GraphNorm official implementation*. GitHub repository: <https://github.com/ljsj2408/GraphNorm>.